FINDIT: CAMPUS LOST & FOUND SYSTEM

Team Onboarding, System Architecture & CI/CD Master Plan (Black & White Edition)

Team Members (5):
Josh • Dom • Rey • Kennedy • Kim
Stack: Docker + Nginx + Flask + PostgreSQL + Jenkins

1. THE #1 THING EVERY MEMBER NEEDS TO KNOW BEFORE WE WRITE CODE

We are NOT graded on building a huge, complicated website. Our professor's guidelines explicitly state: "*The application itself is intentionally kept simple. You are graded mostly on your infrastructure and automation, not on how many features your app has. A small app with a clean, reliable, well-explained pipeline will score higher than a large app that has to be deployed manually.*"

2. WHAT WE ARE BUILDING: "FINDIT"

FindIt is a clean, minimal Campus Lost & Found web application where students and staff can report lost items, post found items, search listings, and submit or review recovery claims. Its real purpose is to serve as a **3-module containerized application** that proves our **Docker architecture** and **automated Jenkins CI/CD pipeline** work end-to-end.

6 Core App Features (What We WILL Build) — IN SCOPE

1. **Report a Lost Item:** Submit item name, category, campus location, date, and description.
2. **Report a Found Item:** Post an item found on campus with location and status (`found`).
3. **Browse & Filter Items:** View all lost/found items and filter by type, status, or keyword.
4. **Submit a Claim:** Submit a claim request on an item with claimant name, student ID, and proof description.
5. **Review Claims (Approve/Reject):** Staff/reviewer button to approve or reject a pending claim.
6. **Auto-Update Status + Build Footer:** Approving a claim marks the item as `returned`. The footer displays the live **Jenkins Build Number** (e.g., `Build #15`).

Features We Will NOT Build (Avoid Bloat) — OUT OF SCOPE

- **No AI image matching** or complex file/photo storage buckets.
- **No real-time chat** or WebSockets between students.
- **No external email/SMS servers** that require third-party API keys.
- **No heavy login/OAuth system** (simple staff verification toggle is enough).
- **No Kubernetes or cloud paid servers** (everything runs locally in Docker).

Why? Every extra feature creates more places for bugs during the live demo without adding a single point to our rubric score.

3. HOW OUR FINAL GRADE IS CALCULATED (AND WHAT "[GATE]" MEANS)

GRADE COMPONENT	WEIGHT	HOW IT WORKS & WHY EVERY MEMBER MUST PARTICIPATE
Group Project Rubric	**70%**	100 points across 9 criteria (plus up to +10 bonus points): • **Jenkins CI/CD Pipeline (20 pts)** & **Automatic Deployment Demo (15 pts)** = 35% of rubric alone! • **Architecture (10)**, **Dockerfiles (10)**, **Docker Compose (10)**, **App Modules (10)**, **Documentation (10)**, **Presentation (10)**, **Teamwork/Git (5)**.
Individual Q&A	**15%**	During the 5-minute Q&A, the instructor asks **each of the 5 members** at least one technical question. Any member can be asked about *any* part of the system (Docker, Nginx, Flask, PostgreSQL, or Jenkins).
Peer Evaluation	**15%**	Groupmates rate each other's contribution (Appendix B), and the instructor **cross-checks this against our Git commit history**. All 5 members must have meaningful commits in GitHub!

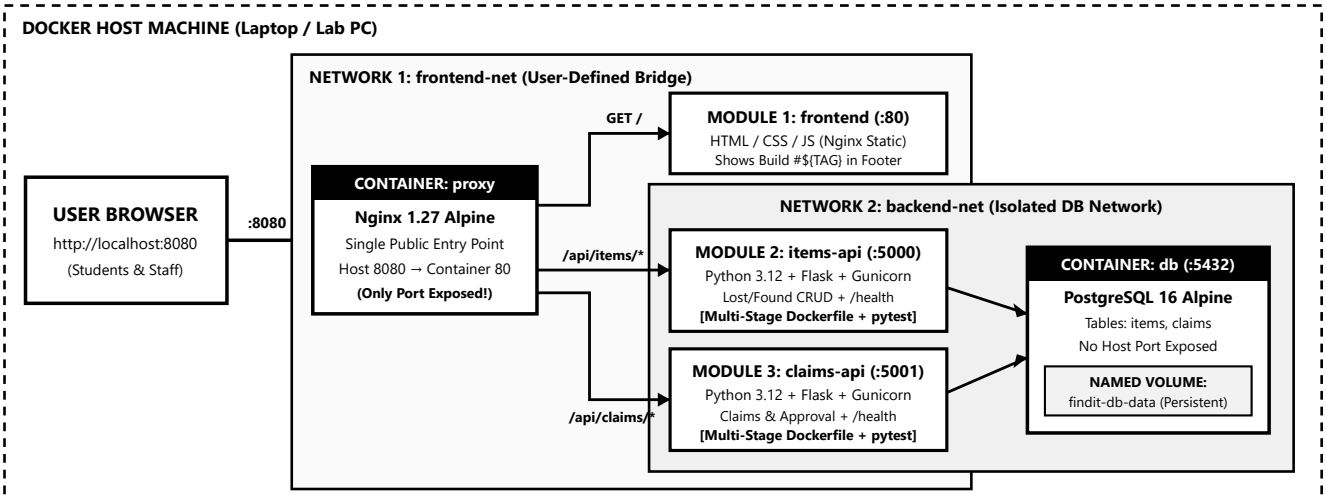
CRITICAL WARNING: WHAT ARE MANDATORY "[GATE]" REQUIREMENTS?

In the guidelines, 7 core items are marked **[GATE]**. If even one **[GATE]** item is missing or broken, our grade for that category is automatically capped at **Beginning (1 out of 4)**. Our architecture guarantees all 7 gates pass:

- **[GATE 1]** At least **3 application modules** in separate containers (`frontend`, `items-api`, `claims-api`).
- **[GATE 2]** Custom `Dockerfile` written by us for every app module.
- **[GATE 3]** One `docker-compose.yml` starts the full app stack.
- **[GATE 4]** PostgreSQL uses a **named volume** so data survives restarts.
- **[GATE 5]** Jenkins runs in Docker with a persistent `jenkins_home` volume (in a separate compose file).
- **[GATE 6]** Declarative `Jenkinsfile` in repo root triggered **automatically** by `git push` (never clicking "Build Now").
- **[GATE 7]** A **failing test stops the pipeline** before the Deploy stage.

4. VISUAL SYSTEM ARCHITECTURE (HOW THE CONTAINERS TALK TO EACH OTHER)

A common mistake is thinking the static Frontend container talks directly to the Backend containers. In reality, the **User's Web Browser** sends all requests to a single public door—**Nginx Reverse Proxy (Port 8080)**—and Nginx routes each URL path to the right container inside Docker's private networks.



5. COMPLETE CONTAINER & TECHNOLOGY BREAKDOWN (WHY WE CHOSE EACH PART)

In Section 4 of the guidelines, our professor requires us to **justify every technology choice** in our documentation and presentation. Here is our exact container lineup and the justification everyone can use during Q&A:

CONTAINER	BASE IMAGE / TECH	INTERNAL PORT & NET	WHAT IT DOES & WHY WE CHOSE IT (Q&A JUSTIFICATION)
1. proxy INFRASTRUCTURE	`nginx:1.27-alpine` Custom `Dockerfile`	`80` (Published as `8080` on host) `frontend-net`	**Single Entry Point & Reverse Proxy:** Routes `/` to `frontend`, `/api/items/` to `items-api`, and `/api/claims/` to `claims-api`. We bake `nginx.conf` into the image so Jenkins never hits host bind-mount path errors.
2. frontend APP MODULE 1	`nginx:1.27-alpine` HTML5, CSS3, ES6 JS	`80` (Internal only) `frontend-net`	**User Interface Container:** Serves the FindIt Lost & Found board. Accepts a Docker build argument (`ARG BUILD_NUMBER`) and injects it into the footer so we can visually prove live deployments and rollbacks.
3. items-api APP MODULE 2	`python:3.12-slim` Flask + Gunicorn + `pytest`	`5000` (Internal only) `frontend-net` & `backend-net`	**Items Microservice:** Handles `GET /api/items`, `POST /api/items`, `PATCH /api/items/<id>`, and `GET /api/items/health`. Uses a **multi-stage Dockerfile** (`base` → `test` → `production`) and runs as a non-root user.
4. claims-api APP MODULE 3	`python:3.12-slim` Flask + Gunicorn + `pytest`	`5001` (Internal only) `frontend-net` & `backend-net`	**Claims Microservice:** Handles submitting claims (`POST /api/claims`), listing claims (`GET /api/claims`), approving/rejecting claims (`PATCH /api/claims/<id>`), and `GET /api/claims/health`. Updates item status to `returned` on approval.
5. db INFRASTRUCTURE	`postgres:16-alpine` + `db/init.sql`	`5432` (Internal only) `backend-net` only	**Persistent Relational Database:** Stores `items` and `claims` tables with foreign-key integrity. Uses named volume `findit-db-data` so data survives container restarts. Isolated on `backend-net` so `proxy` and `frontend` cannot reach it directly.
6. jenkins SEPARATE COMPOSE	`jenkins/jenkins:lts-jdk17` + Docker CLI installed	`8080` (Published as `8081` on host)	**CI/CD Server:** Runs in `infra/jenkins/docker-compose.yml` with persistent volume `jenkins_home`. Kept separate from the app stack so the pipeline never restarts or redeploys Jenkins itself.

5 TECHNICAL UPGRADES WE MADE OVER CHATGPT'S INITIAL DRAFT (WHY OUR BUILD WON'T CRASH)

1. **Multi-Stage Dockerfiles for Testing:** Jenkins runs inside a Java container without Python installed. By adding a `FROM base AS test` stage to `items-api/Dockerfile` and `claims-api/Dockerfile`, Jenkins runs `docker build --target test` inside Docker with zero host dependencies.
2. **Isolated Unit Tests vs. Live Smoke Tests:** During `docker build --target test`, PostgreSQL is not connected. Our `pytest` unit tests validate API logic using an isolated test fixture, while the **Smoke Test stage** after `Deploy` tests the live containers + live PostgreSQL DB.
3. **Baked-in Nginx Config (`proxy/Dockerfile`):** Avoids Docker-in-Docker bind-mount path failures when Jenkins runs `docker compose up`.
4. **Per-Module `.dockerignore` Files:** Placed inside `frontend/`, `items-api/`, and `claims-api/` as required by Rubric Criterion #2.
5. **Network Segmentation (`frontend-net` + `backend-net`):** Gives us maximum marks on Rubric Criterion #1 & #3.

6. HOW OUR AUTOMATED JENKINS PIPELINE WORKS (35% OF RUBRIC GRADE)

Whenever any team member commits a change and runs `git push origin main`, GitHub notifies Jenkins automatically (via **Webhook + ngrok**, with **Poll SCM** `H/2 * * * *` configured as a backup fallback). Jenkins reads our root `Jenkinsfile` and runs these **5 mandatory stages**:

STAGE 1
Checkout
Pulls latest commit from our shared GitHub repo using `checkout scm`.

→

STAGE 2 [GATE]
Test
Runs `docker build --target test` on both APIs. **Stops pipeline if any test fails!**

→

STAGE 3
Build Images
Runs `docker compose build` with `TAG=${BUILD_NUMBER}`.

→

STAGE 4
Deploy
Injects secret `.env` from Jenkins Credentials & runs `docker compose up -d`.

→

STAGE 5
Smoke Test
Pings `/api/items/health` & `/api/claims/health` via Nginx proxy.

Scenario A: Normal Push (All Tests Pass) — **PASS → LIVE**

1. Dom updates the heading in `frontend/index.html` and runs `git push`.
2. Jenkins starts **Build #15** automatically within seconds.
3. `Checkout` → `Test` (all `pytest` checks pass) → `Build Images` tags images as `:15` and bakes `Build #15` into the frontend footer.
4. `Deploy` recreates changed containers while PostgreSQL keeps all existing lost & found records in `findit-db-data`.
5. `Smoke Test` confirms HTTP 200 on all health endpoints. Browser refresh shows the new heading and **Build #15**!

Scenario B: Broken Code Push (Quality Gate) — **FAIL → BLOCKED**

1. During the live demo, we push a commit that breaks a rule or test in `items-api`.
2. Jenkins starts **Build #16** automatically.
3. At **Stage 2 (`Test`)**, `pytest` catches the bug and returns exit code `1`.
4. **Jenkins immediately aborts Build #16!** Stages 3, 4, and 5 are skipped completely.
5. We refresh the browser at `http://localhost:8080`: the site is **still up and running Build #15** with zero downtime!

7. HOW SECRETS, PERSISTENCE, AND ROLLBACK WORK

MECHANISM	HOW WE IMPLEMENT IT (AND PROVE IT LIVE TO THE PROFESSOR)
Secret Management (No Passwords in Git)	We commit only `.env.example` (with placeholder values) to GitHub, and list `.env` in `.gitignore`. The real `.env` is stored inside **Manage Jenkins → Credentials** as a *Secret file* (ID: `findit-env`). During the `Deploy` stage, Jenkins copies `$ENV_FILE` to `.env`, starts the containers, and deletes `.env` in the `post { always { sh 'rm -f .env' } }` block.
Database Persistence (Survives Redeploys)	PostgreSQL stores its data at `/var/lib/postgresql/data` mapped to the named Docker volume `findit-db-data`. When Jenkins deploys a new build (or when we run `docker compose down` and `docker compose up -d` without `-v`), all reported Lost & Found items and claims remain intact.
One-Command Rollback (Image Tagging)	Every successful pipeline run tags images with the Jenkins build number: `findit/frontend:${TAG}`, `findit/items-api:${TAG}`, `findit/claims-api:${TAG}`. To roll back from Build #15 to Build #14 instantly without rebuilding: `TAG=14 docker compose -p findit up -d --no-build` (the website footer immediately reverts to **Build #14**!).

8. AUTOMATED TESTS WE WILL RUN IN STAGE 2 (`PYTEST`)

To earn **4/4 Excellent** on Rubric Criterion #6 ("*automated tests on all backend modules*"), both `items-api` and `claims-api` include focused `pytest` test suites that run inside Docker during every build:

`items-api/tests/test_items.py` — **4 TESTS**

- `test_health_endpoint`: Verifies `GET /api/items/health` returns `200 OK` and JSON `{"status": "healthy"}`.
- `test_create_item_valid`: Verifies posting a valid lost/found item returns `201 Created`.
- `test_create_item_missing_fields`: Verifies missing title/category/type is rejected with `400 Bad Request`.
- `test_list_and_filter_items`: Verifies `GET /api/items?type=lost` returns matching items.

`claims-api/tests/test_claims.py` — **4 TESTS**

- `test_health_endpoint`: Verifies `GET /api/claims/health` returns `200 OK`.
- `test_submit_claim_nonexistent_item`: Verifies claiming a missing item returns `404 Not Found`.
- `test_prevent_claim_on_returned_item`: Verifies claiming an already-returned item is rejected with `400 Bad Request`.
- `test_approve_claim_updates_status`: Verifies approving a claim marks the claim `approved` and the item `returned`.

9. OUR 5 TEAM MEMBERS: ROLES, RESPONSIBILITIES & GIT OWNERSHIP

Remember: **all 5 members must have meaningful commits in our GitHub repository** (using feature branches and Pull Requests), and **all 5 members must speak during the 10-minute presentation**. Here is how our roles align with Section 3 of the professor's guidelines:

MEMBER & ROLE	PRIMARY TECHNICAL & PROJECT RESPONSIBILITIES	GIT FILES & DELIVERABLES OWNED
1. Josh *Project Lead / Scrum Master*	• Leads project timeline, checkpoints, and team coordination. • Manages GitHub repository settings, branch protection, and PR reviews. • Co-designs the system architecture diagram & coordinates full-stack rehearsals. • Leads the presentation intro, architecture overview, and demo flow.	• **Appendix A Proposal** • `docs/architecture.png` • `.gitignore`, PR reviews • Task board & Contribution Log
2. Dom *Frontend, QA & Documentation Lead*	• Builds the **FindIt Frontend** (HTML, CSS, Vanilla JS) with lost/found forms, search/filter bar, claim modal, and live `Build #` footer. • Connects UI to `/api/items` and `/api/claims` via Nginx. • Writes pipeline smoke verification checks & leads the `README.md`, Technical Documentation PDF (8–15 pages), and Presentation Slides.	• `frontend/index.html` • `frontend/css/styles.css` • `frontend/js/app.js` • `README.md`, `docs/` PDF & Slides
3. Rey *Backend & Database Engineer*	• Designs PostgreSQL schema & seed data (`items` and `claims` tables). • Builds **Module 2 (`items-api`)** and **Module 3 (`claims-api`)** in Python Flask. • Implements `/api/items/health` and `/api/claims/health` endpoints. • Writes the `pytest` automated unit test suites for both backend APIs.	• `db/init.sql` • `items-api/app.py` & `tests/` • `claims-api/app.py` & `tests/` • `requirements.txt` files
4. Kennedy *Infrastructure & Docker Engineer*	• Writes multi-stage `Dockerfile`s (`base`, `test`, `production`, non-root user, `HEALTHCHECK`) and `.dockerignore` for each module. • Configures Nginx reverse proxy (`proxy/nginx.conf` + `proxy/Dockerfile`). • Writes root `docker-compose.yml` (networks, named volume, healthcheck dependencies) and `.env.example`.	• `docker-compose.yml`, `.env.example` • `proxy/Dockerfile` & `nginx.conf` • `frontend/Dockerfile` • `items-api/Dockerfile`, `claims-api/Dockerfile` & `.dockerignore`s
5. Kim *DevOps / CI-CD Engineer*	• Sets up containerized Jenkins in `infra/jenkins/` with Docker CLI and persistent `jenkins_home` volume. • Configures Jenkins Credentials (`findit-env`) & GitHub Webhook (ngrok) / Poll SCM. • Writes the declarative root `Jenkinsfile` (all 5 stages, image tagging, quality gate, smoke test, and `post` cleanup) + rollback script.	• `Jenkinsfile` • `infra/jenkins/Dockerfile` • `infra/jenkins/docker-compose.yml` • Webhook / Trigger & Rollback setup

10. REQUIRED REPOSITORY FOLDER STRUCTURE (ALIGNED WITH SECTION 8 OF PDF)

```
FindIt/
├── Jenkinsfile                  # Declarative CI/CD pipeline (Kim)
├── docker-compose.yml           # Full app stack orchestration (Kennedy)
├── .env.example                 # Committed placeholder env vars (Kennedy)
├── .gitignore                   # Ignores .env, __pycache__, etc. (Josh)
├── README.md                    # Setup, architecture & demo guide (Dom)
├── docs/
│   ├── architecture.png         # System architecture diagram (Josh)
│   └── technical-documentation.pdf  # 8-15 page final report (Dom & All)
├── proxy/
│   ├── Dockerfile               # Bakes nginx.conf into image (Kennedy)
│   └── nginx.conf               # Routes /, /api/items/, /api/claims/
├── frontend/
│   ├── Dockerfile               # Injects BUILD_NUMBER into UI (Kennedy)
│   ├── .dockerignore            # Excludes unnecessary files
│   ├── index.html               # FindIt Lost & Found UI (Dom)
│   ├── css/styles.css           # Responsive styling (Dom)
│   └── js/app.js                # Fetch calls to /api/* (Dom)
├── items-api/
│   ├── Dockerfile               # Multi-stage: base -> test -> prod (Kennedy)
│   ├── .dockerignore            # Excludes .venv, __pycache__, .git
│   ├── requirements.txt         # Flask, gunicorn, psycopg2-binary, pytest
│   ├── app.py                   # Items CRUD + GET /api/items/health (Rey)
│   └── tests/
│       └── test_items.py        # Automated pytest unit tests (Rey)
├── claims-api/
│   ├── Dockerfile               # Multi-stage: base -> test -> prod (Kennedy)
│   ├── .dockerignore            # Excludes .venv, __pycache__, .git
│   ├── requirements.txt         # Flask, gunicorn, psycopg2-binary, pytest
│   ├── app.py                   # Claims CRUD + GET /api/claims/health (Rey)
│   └── tests/
│       └── test_claims.py       # Automated pytest unit tests (Rey)
├── db/
│   └── init.sql                 # Creates items & claims tables + seed data (Rey)
└── infra/
    └── jenkins/
        ├── Dockerfile           # jenkins:lts-jdk17 + docker-ce-cli (Kim)
        └── docker-compose.yml   # Separate Jenkins stack on :8081 (Kim)
```

11. OUR 8-PHASE STEP-BY-STEP ROADMAP

PHASE	LEAD(S)	WHAT WE DO & CHECKPOINT GOAL
Phase 0–1	Josh & All	Verify Git & Docker Desktop work on our machines. Submit **Appendix A Proposal** & create shared GitHub repo.
Phase 2	Rey, Dom, Kennedy	Build `frontend`, `items-api`, and `claims-api` with `/health` routes, `pytest` suites, and multi-stage `Dockerfile`s.
Phase 3	Kennedy & Rey	Wire up `proxy`, `db` (`init.sql` + `findit-db-data` volume), and `docker-compose.yml`. Verify `docker compose up -d --build` works at `http://localhost:8080`.
Phase 4–5	Kim & Kennedy	Start Jenkins in `infra/jenkins/` (`:8081`), add `findit-env` secret file in Jenkins Credentials, and test root `Jenkinsfile`.
Phase 6–7	Kim, Rey, Dom	Connect GitHub Webhook (ngrok) / Poll SCM. Test auto-deploy, failing-test quality gate, DB persistence, and image tag rollback.
Phase 8	Dom, Josh & All	Finalize `README.md`, 8–15 page Technical Documentation PDF, 12–20 Slides, rehearse 10-min live demo, and record backup video.

12. MANDATORY 7-STEP LIVE DEMONSTRATION SCRIPT (10 MINUTES + 5 MINS Q&A)

Section 10 of the guidelines requires every group to demonstrate these **exact 7 steps in order** (and all 5 members must speak):

1. **Show Running Stack & Build Number (Kennedy):** Run `docker compose ps`, open `http://localhost:8080` through Nginx proxy, point out the current build label in the footer (e.g., `Build #14`), and submit a new Lost Item + Claim.
2. **Push a Visible Code Change (Dom):** Edit the hero banner text in `frontend/index.html`, commit, and run `git push`.
3. **Show Automatic Pipeline Trigger (Kim):** Hands off the mouse—show Jenkins starting **Build #15 automatically** from the GitHub webhook/poll and walk through `Checkout → Test → Build Images → Deploy → Smoke Test`.
4. **Show Live Change in Browser (Dom):** Refresh `http://localhost:8080`—show the updated banner text and `Build #15` in the footer.
5. **Prove the Quality Gate with a Failing Test (Rey):** Push a commit that intentionally breaks a test in `items-api`. Show Build #16 failing at the **Test** stage while the live app stays up on Build #15. Then `git revert` and push so Build #17 passes.
6. **Prove Database Persistence (Rey):** Point out that the Lost Item and Claim created in Step 1 are still present in PostgreSQL after multiple redeployments (stored in the `findit-db-data` named volume).
7. **Prove Instant Rollback (Kim & Josh):** Run `TAG=14 docker compose -p findit up -d --no-build` and refresh the browser to show the footer back on `Build #14`.

13. INDIVIDUAL Q&A CHEAT SHEET (15% OF GRADE — EVERYONE MUST READ THIS!)

Q1: Why mount `/var/run/docker.sock` in Jenkins? Is it safe?

Answer (From Page 9 Security Note): Mounting the host's Docker socket as root lets our Jenkins container build and start sibling containers on the host easily in a classroom lab. However, in real production it is a security risk because it gives Jenkins root-equivalent control over the host machine. Safer production alternatives are **Kaniko**, **rootless Docker**, **Docker-in-Docker (DinD) with TLS**, or dedicated ephemeral build agents.

Q2: How do your unit tests run if Jenkins doesn't have Python?

Answer: Our backend `Dockerfile`s use **multi-stage builds** (`base`, `test`, and `production`). In the `Test` stage of the `Jenkinsfile`, Jenkins runs `docker build --target test`, which executes `pytest` inside an isolated Python container. Our production image stage excludes test tools and runs as a non-root user.

Q3: Why is only port 8080 exposed, and how does DB data survive?

Answer: Only the `proxy` (Nginx) container publishes port `8080:80` to the host so all external traffic passes through a single entry point. PostgreSQL is isolated on `backend-net` with no host port exposed, and its `/var/lib/postgresql/data` folder is mounted to the `findit-db-data` **named volume** so records persist across container removals.

Q4: Where are database passwords stored? Are they in Git?

Answer: Never in Git! Only `.env.example` with placeholder values is committed, and `.env` is in `.gitignore`. The real production `.env` is stored encrypted in **Jenkins Credentials** as a Secret File (`findit-env`), injected during the `Deploy` stage via `withCredentials`, and deleted in `post { always { rm -f .env } }`.